Gestion des livraisons

Préparation du chargement d'un camion

Rapport des choix algorithmiques — Python 3 — octobre 2026

1. Problème et langage

Une entreprise de livraison prépare chaque jour le chargement de ses camions. Elle dispose de colis décrits par un identifiant, un poids, une valeur, une priorité et une destination. Le programme doit gérer une collection qui peut aller jusqu'à 100 000 colis, retrouver un colis par son identifiant, les afficher selon le poids, la valeur ou la priorité, puis choisir un sous-ensemble qui tient dans un camion de capacité limitée. Ce sous-ensemble doit maximiser un critère, et non pas seulement respecter la capacité. La priorité doit pouvoir influencer ce choix.

Le langage retenu est Python 3 (le programme vise Python 3.10 ou plus récent ; les mesures ont été faites avec Python 3.12.10 sous Windows). Ce choix ne cherche pas un avantage à l'évaluation : l'énoncé précise qu'il n'y en a pas. Python est retenu parce que son dict est une table de hachage, adaptée à la recherche par identifiant, et parce que le code reste lisible au moment d'expliquer un algorithme. Le tri fusion et le sac à dos sont écrits dans le projet. Aucune bibliothèque externe ne résout ces deux problèmes. Le menu, les mesures de temps et le générateur aléatoire viennent de la bibliothèque standard. La fonction sorted n'est appelée que dans les mesures, comme point de comparaison : elle n'organise pas les colis.

2. Modélisation

Un colis est un objet immuable : identifiant entier, poids entier en kilogrammes (au moins 1), valeur entière positive ou nulle, priorité entière au moins égale à 1, destination non vide. Les poids entiers sont ceux de l'énoncé. Ils rendent possible la programmation dynamique. L'identifiant est unique : un second colis avec le même identifiant est refusé.

Hypothèse sur la priorité, car l'énoncé ne fixe pas le sens de l'échelle : un nombre plus grand est une priorité plus importante. 5 compte donc plus que 1. Les exemples utilisent les valeurs 1 à 5, mais le programme accepte toute priorité supérieure ou égale à 1. L'hypothèse inverse (1 = plus urgent) serait aussi cohérente si elle était écrite de la même façon. Ce n'est pas celle retenue ici.

La collection est un dict indexé par l'identifiant. L'ordre d'insertion est conservé par le langage, et c'est cet ordre que le menu affiche avant tout tri. Les affichages triés sont des vues : ils ne remplacent pas la collection.

3. Choix algorithmiques

Quatre décisions structurent le programme. Les trois premières portent sur des algorithmes. La quatrième porte sur la règle qui mélange valeur, poids et priorité. Pour chacune, le problème, la solution, les complexités, une alternative et la raison du choix sont séparés, comme demandé.

3.1 Retrouver un colis par son identifiant

Élément	À présenter
Problème	Ajouter un colis et le retrouver à partir de son identifiant, dans une collection qui peut contenir 100 000 éléments. L'identifiant est unique.
Solution	Table de hachage : le dict de CPython. La clé est l'identifiant, la valeur est le colis. CPython range les clés par adressage ouvert. L'ajout vérifie d'abord que l'identifiant est libre.
Temps	O(1) en moyenne pour l'ajout et pour la recherche, que l'identifiant existe ou non. Le pire cas théorique d'une table de hachage est O(n), si toutes les clés tombaient dans le même paquet. Avec des identifiants entiers, ce cas ne se produit pas. Le redimensionnement est amorti : insérer n colis coûte O(n).
Espace	O(n) pour les colis et pour la table.
Alternative	Une liste parcourue linéairement : recherche en O(n). Ou une liste triée par identifiant avec une dichotomie : recherche en O(log n), mais insertion en O(n) a cause du décalage des éléments. Un arbre équilibré donnerait O(log n) partout, avec une constante plus lourde et plus de code, pour une opération qui n'a pas besoin d'être ordonnée.
Justification	La recherche par identifiant est une action du menu, répétée. Elle doit rester rapide quand n augmente. Les mesures restent autour de 0,07 microseconde de 100 à 100 000 colis. La table de hachage est adaptée. Une structure plus compliquée n'apporterait rien sur cette opération.

3.2 Afficher les colis selon un critère

Élément	À présenter
Problème	Afficher les colis selon le poids, la valeur ou la priorité, en ordre croissant ou décroissant.
Solution	Tri fusion ascendant (bottom-up), écrit dans src/tri.py. Les clés sont calculées une seule fois, puis chaque passe fusionne des sous-suites de taille 1, 2, 4, et ainsi de suite. Le tri est stable : à critère égal, l'élément le plus à gauche reste devant, donc l'ordre d'insertion est gardé. Pour l'ordre décroissant, on trie selon l'opposé de la clé, ce qui préserve cette stabilité. La collection n'est pas modifiée : le tri renvoie une vue.
Temps	O(n log n) dans tous les cas. Il y a log2(n) passes, et chaque passe déplace n éléments. La fonction de clé est appelée n fois, pas à chaque comparaison.
Espace	O(n) : la copie, le tableau tampon et les deux tableaux de clés. La version ascendante n'utilise pas de pile d'appels. L'espace supplémentaire lié à la récursion est donc O(1).
Alternative	Le tri rapide : O(n log n) en moyenne, O(n^2) dans le pire cas, et instable. Le Timsort de CPython, appelé par sorted : même ordre de grandeur, constante bien plus petite parce qu'il est écrit en C. Des index triés maintenus à chaque ajout : intéressants seulement si l'on triait en permanence. Ici l'affichage est ponctuel, et il coûte déjà O(n) rien que pour parcourir les colis.
Justification	On veut une borne garantie, un tri stable, et un algorithme que l'on peut expliquer pas à pas. À 100 000 colis, le tri fusion prend 143 ms. C'est assez rapide pour une demande d'affichage. Le tri natif prend 8,9 ms sur le même lot, soit environ 16 fois moins. Ce serait l'amélioration à prendre si le tri devenait très fréquent. Tant qu'il reste ponctuel, recalculer la vue est plus simple que de maintenir trois index, et 143 ms restent acceptables.

3.3 Charger le camion

Chaque colis est pris au plus une fois. C'est un sac à dos 0/1. Le programme ne s'arrête pas au premier sous-ensemble qui tient dans le camion : il maximise un score entier. Trois méthodes sont branchées selon la taille du problème, parce qu'une seule méthode ne reste pas raisonnable pour toutes les tailles.

Élément	À présenter
Problème	Choisir un ensemble de colis dont le poids total est inférieur ou égal à la capacité C du camion, en maximisant soit la somme des valeurs, soit la somme des scores valeur × priorité.
Solution	1) Programmation dynamique si n × (C + 1) est inférieur ou égal à 8 000 000. dp[w] est le meilleur score atteignable avec une capacité de w kilogrammes. Les capacités sont parcourues à l'envers, donc un colis ne peut pas être pris deux fois. Une table d'un octet par couple (colis, capacité) sert à reconstruire la liste retenue. 2) Sinon, rencontre au milieu si n est inférieur ou égal à 36. Les deux moitiés sont énumérées. La seconde est triée par poids avec le tri fusion, un préfixe retient le meilleur score, et une dichotomie trouve le complément qui tient encore. 3) Sinon, glouton : tri par densité score / poids, puis on prend chaque colis s'il reste de la place.
Temps	Programmation dynamique : O(n × C). Rencontre au milieu : O(n × 2^(n/2)), a cause de l'énumération et du tri des sous-ensembles. Glouton : O(n log n) pour le tri, puis O(n) pour le remplissage. Les colis plus lourds que le camion sont écartés avant le calcul : ils n'appartiennent à aucune solution réalisable.
Espace	Programmation dynamique : O(n × C) pour la table de décisions, plus O(C) pour la ligne de scores. Rencontre au milieu : O(2^(n/2)). Glouton : O(n).
Alternative	La force brute en O(2^n) ne dépasse pas une vingtaine de colis. Le glouton seul est simple, mais le contre-exemple plus bas montre qu'il n'est pas toujours optimal. Un schéma d'approximation (FPTAS) garantirait une valeur proche de l'optimum quand n et C sont tous les deux grands. Le branch and bound peut être rapide sur certaines instances, sans borne de temps exploitable ici. Ces deux pistes sont laissées de côté : sur les tailles et la capacité de l'énoncé, la programmation dynamique est déjà exacte et rapide.
Justification	Les poids du sujet sont des entiers, et la capacité d'exemple vaut 50 kg. Dans ce cadre, la programmation dynamique reste l'algorithme utilisé de 100 à 100 000 colis. À 100 000 colis elle prend 120 ms et environ 8,5 Mo. La rencontre au milieu couvre l'autre cas utile : peu de colis, mais une capacité trop grande pour la table, tout en restant exacte. Le glouton n'est la que pour répondre quand les deux méthodes exactes deviennent démesurées. Le menu affiche alors que le résultat n'est pas garanti optimal.

La programmation dynamique et la rencontre au milieu sont toujours optimales pour le critère demandé (valeur, ou score). Le glouton ne l'est pas. Contre-exemple, capacité 50 kg : un colis de 10 kg et de valeur 60, un de 20 kg et de valeur 100, un de 30 kg et de valeur 120. Leurs densités valent 6, 5 et 4. Le glouton prend les deux premiers, valeur 160, et le troisième ne rentre plus. L'optimum vaut 220 : les colis de 20 kg et de 30 kg. Le programme donne bien 220 par programmation dynamique, et 160 par le glouton.

Le cœur de la programmation dynamique tient en quelques lignes. Le parcours à l'envers est ce qui empêche de réutiliser le même colis :

```
pour chaque colis, de poids p et de score s :
    pour w allant de C jusqu'à p, en descendant :
        si dp[w - p] + s > dp[w] :
            dp[w] = dp[w - p] + s
            retenir que ce colis est pris pour la capacité w
```

On ne remplace dp[w] qu'en cas de score strictement meilleur. Deux solutions de même score sont toutes les deux optimales ; en cas d'égalité, le programme garde celle qui prenait le moins de colis arrivés plus tôt. La reconstruction part de la capacité maximale. Comme les scores sont positifs ou nuls, dp[C] est le meilleur score : remplir moins que C ne peut pas faire mieux.

3.4 Faire intervenir la priorité

Élément	À présenter
Problème	Favoriser les colis de priorité importante, tout en tenant compte du poids et de la valeur. L'énoncé n'impose pas de formule. Deux règles différentes sont acceptables si elles sont cohérentes et expliquées.
Solution	On maximise la somme des scores valeur × priorité, sous la contrainte de poids. La valeur reste le gain de base. La priorité est un multiplicateur : à valeur égale, un colis de priorité 5 compte cinq fois plus qu'un colis de priorité 1. Le poids n'entre pas dans le score. Il reste la contrainte du camion, et, pour le glouton seulement, le diviseur de la densité. Le menu propose deux modes. Le mode valeur ignore la priorité. Le mode score l'utilise. Les deux appellent le même sac à dos, avec un score différent.
Temps	Le même que la méthode de chargement retenue. Le score d'un colis se calcule en O(1). Passer de la valeur au score ne change pas la complexité.
Espace	Le même que la méthode de chargement. Aucune structure supplémentaire.
Alternative	Charger d'abord les plus fortes priorités, puis optimiser la valeur dans la place restante. C'est simple, mais un seul colis prioritaire et lourd peut bloquer plusieurs colis dont le score cumulé est meilleur. Autre formule : valeur + α × priorité. Elle additionne deux unités différentes, et α serait arbitraire. Intégrer le poids dans le score le sortirait de la contrainte, alors que le camion a une capacité dure.
Justification	Le produit donne un objectif additif, à valeurs entières. La programmation dynamique et la rencontre au milieu restent donc optimales pour cet objectif. Elles peuvent refuser un colis très prioritaire s'il occupe une place qui vaudrait un meilleur score total avec d'autrès colis. La solution n'est pas optimale pour la valeur monétaire. C'est le compromis cherché : la priorité influence la sélection, sans devenir un ordre absolu qui ignorerait la valeur et le poids.

Exemple, capacité 10 kg. A : 8 kg, valeur 20, priorité 5, score 100. B : 6 kg, valeur 25, priorité 1, score 25. C : 5 kg, valeur 18, priorité 4, score 72. D : 4 kg, valeur 14, priorité 3, score 42.

En mode valeur, le programme charge B et D : poids 10 kg, valeur 39. Aucun autre couple ne fait mieux (A laisse trop peu de place, C et D ne valent que 32). En mode score, il charge C et D : poids 9 kg, score 114, valeur monétaire 32 seulement. A seul ne fait que 100. La priorité change donc bien la sélection. Les deux résultats sont optimaux, chacun pour son critère. Le second n'est pas optimal pour la valeur, et le programme ne prétend pas le contraire : l'écran affiche le critère, le score, la valeur, et si l'algorithme utilisé est optimal pour ce critère.

Sur l'exemple du sujet (colis 101 à 105, poids total 30 kg, valeur 110), un camion de 50 kg emporte tout le lot. Le score vaut 20 × 2 + 10 × 5 + 40 × 1 + 25 × 4 + 15 × 3 = 275. Le cas est volontairement simple : il vérifie que le programme n'écarte rien quand tout rentre.

4. Comportement quand le nombre de colis augmente

Les mesures portent sur 100, 1 000, 10 000 et 100 000 colis, avec une graine fixe (42) pour pouvoir les refaire. Poids tiré entre 1 et 30 kg, valeur entre 1 et 100, priorité entre 1 et 5. Le camion des mesures a une capacité de 50 kg, comme dans l'exemple. La recherche est une moyenne sur 5 000 requêtes, moitié sur un identifiant présent, moitié sur un identifiant absent. Les temps n'incluent pas l'affichage : écrire 100 000 lignes à l'écran dominerait le tri, et le menu paginé de 20 en 20 justement pour eviter cela. La mémoire est un pic d'allocations pendant l'opération, mesuré à part, parce que l'outil de traçage ralentit l'exécution. Une seule exécution, Python 3.12.10, Windows. Les ordres de grandeur comptent plus que le dixième de milliseconde.

n	Ajout ms	Rech. µs	Tri ms	Natif ms	Camion ms	Tri Ko	Camion Ko
100	0,01	0,062	0,07	0,01	0,16	3,4	11,7
1 000	0,07	0,066	0,84	0,08	1,27	32,5	93,1
10 000	0,52	0,065	10,04	0,78	12,20	318	893
100 000	6,89	0,075	143,28	8,91	120,36	3 126	8 742

Pour ces quatre tailles et cette capacité de 50 kg, le chargement utilise à chaque fois la programmation dynamique. Il reste donc optimal. « Rech. µs » est le temps moyen d'une recherche en microsecondes. « Natif ms » est le temps de sorted sur le même critère : ce n'est pas l'algorithme du programme, c'est la comparaison.

4.1 Tri fusion : le coût augmente

Complexité : O(n log n) en temps, O(n) en espace. De 10 000 à 100 000 colis, n est multiplié par 10 et le temps passe de 10,04 ms à 143 ms, soit un facteur d'environ 14. Le facteur théorique de n log n entre ces deux tailles vaut environ 12,5. L'écart est normal : il reste une part fixe, et la constante du tri écrit en Python n'est pas celle d'un tri en C. La mémoire du tri passe de 318 Ko à 3 126 Ko, facteur 9,8 pour un facteur 10 sur n. C'est bien linéaire, conforme au O(n) annoncé.

Le choix reste adapté à 100 000 colis : 143 ms, c'est une fraction de seconde au moment ou l'utilisateur demande une organisation. Il ne serait plus agréable si l'on triait sans cesse, ou si n passait à plusieurs millions. L'amélioration mesurée est le Timsort natif : 8,9 ms au lieu de 143 ms à 100 000 colis, environ 16 fois plus rapide, pour la même complexité O(n log n). On ne l'utilise pas pour organiser les colis, parce que le tri est ici un cœur algorithmique et qu'il doit rester explicable. Si le tri devenait un goulot, ce remplacement serait le bon : même résultat, constante bien plus faible. Maintenir un index trié à jour ne vaudrait le coup que si les affichages triés étaient beaucoup plus fréquents que les ajouts.

4.2 Préparation du camion : le coût augmente

Complexité, dans le régime mesuré : O(n × C) en temps et O(n × C) en espace, avec C = 50. Quand C est fixe, cela se comporte comme O(n). Les temps 0,16 ms, 1,27 ms, 12,20 ms et 120 ms sont multipliés par environ 8, puis 10, puis 10 à chaque fois que n est multiplié par 10. À partir de 1 000 colis, le facteur colle à 10. La mémoire passe de 893 Ko à 8 742 Ko entre 10 000 et 100 000, facteur 9,8. La table de décisions a elle seule occupe n × 51 octets, soit environ 5 Mo à 100 000 colis. Le pic mesuré est un peu au-dessus, parce qu'il compte aussi les listes temporaires de poids et de scores.

Le choix reste adapté sur tout l'intervalle demandé, pour un camion de 50 kg : 120 ms, solution optimale, mémoire d'environ 8,5 Mo. Il cesserait de l'être si la capacité devenait grande en même temps que n. À titre d'ordre de grandeur, 100 000 colis et une capacité de 5 000 kg demanderaient de l'ordre de 500 millions de cases : plusieurs secondes au minimum, et une mémoire qui dépasse le seuil de 8 millions de cases. Le programme bascule alors. S'il reste au plus 36 colis, la rencontre au milieu donne encore l'optimum, avec un coût qui ne dépend plus de C. Au-delà, le glouton répond vite mais sans garantie. L'amélioration suivante, si l'on voulait rester proche de l'optimum dans ce dernier cas, serait un schéma d'approximation. Il n'est pas implémenté : pour la capacité et les tailles de l'énoncé, la solution exacte tient déjà largement.

4.3 Recherche et ajout : le coût unitaire ne suit pas n

La recherche reste entre 0,062 et 0,075 microseconde alors que n est multiplié par 1 000. La légère hausse à 100 000 colis est un effet de cache, pas un changement de complexité. C'est le comportement attendu d'une table de hachage, et c'est pour cela que ce choix reste le bon. Il n'y a pas d'amélioration utile à proposer sur la recherche.

Le temps d'insertion des n colis passe de 0,01 ms à 6,89 ms. C'est le coût total de n ajouts, donc une croissance linéaire, cohérent avec un ajout amorti en O(1). Un ajout isolé reste négligeable à 100 000 colis. Le choix reste adapté.

5. Utilisation de l'ensemble

Le menu relie les opérations sans les recalculer en cachette. Afficher montre l'ordre d'insertion. Organiser lance le tri fusion et affiche son temps. Rechercher interroge la table de hachage. Préparer un camion demande la capacité et le critère, puis conserve le plan jusqu'au prochain calcul. Charger un nouveau jeu de données efface ce plan, parce qu'il ne porterait plus sur les mêmes colis. Ajouter un colis ne modifie pas un plan déjà calculé : la préparation est une décision explicite. Les statistiques (effectifs, poids, valeur, destinations, priorités) se calculent en un parcours, donc en O(n).

6. Bilan

La collection est une table de hachage, parce que la recherche par identifiant doit rester en temps constant jusqu'à 100 000 colis, et les mesures le confirment. Les affichages ordonnés passent par un tri fusion stable de coût garanti O(n log n), assez rapide à cette taille, avec le tri natif comme amélioration mesurée si la constante devenait gênante. Le camion est un sac à dos 0/1. Sur les tailles demandées et une capacité de quelques dizaines de kilogrammes, la programmation dynamique donne une solution optimale en une fraction de seconde. La priorité entre par le score valeur × priorité, hypothèse écrite et réversible dans son principe, pour laquelle cette solution reste optimale. Le glouton n'est utilisé que lorsque l'exactitude deviendrait démesurée, et il est alors présenté comme tel.